

2 • Neural ODEs ⇄

A **Neural ODE** replaces the right-hand side of a differential equation with a neural network:

$$\dot{u} = f_{\theta}(u, t)$$

where f_{θ} is a small neural network with trainable parameters θ . Training means: simulate the ODE forward, compare to data, and backpropagate *through the ODE solver* to update θ . This notebook trains a neural ODE to reproduce a 2D spiral trajectory, following the classic example from Chen et al. (2018), *Neural Ordinary Differential Equations*.

```
1 using Lux, OrdinaryDiffEq, SciMLSensitivity, Optimization, OptimizationOptimisers,  
   ComponentArrays, Random, Plots
```

1. Generate synthetic "ground truth" data ⇄

We pretend we don't know the true dynamics and only observe noiseless samples of a 2D spiral produced by a simple linear ODE $\dot{u} = Au$.

Error message

Detected non-constant types in an out-of-place ODE solve, i.e. for `'du = f(u,p,t)'` we see `'typeof(du) != typeof(u/t)'`. This is not supported by OrdinaryDiffEq.jl's solvers. Please either make `'f'` type-constant (i.e. `typeof(du) == typeof(u/t)`) or use the mutating in-place form `'f(du,u,p,t)'` (which is type-constant by construction).

Note that one common case for this is when computing with GPUs, using `'Float32'` for `'u0'` and `'Float64'` for `'tspan'`. To correct this, ensure that the element type of `'tspan'` matches the preferred compute type, for example `'ODEProblem(f,0f0,(0f0,1f0))'` for `'Float32'`-based time.

```
typeof(u/t) = Vector{Float32}
typeof(du) = Vector{Float64}
```

Show stack trace...

```
1 begin
2     rng = Random.default_rng()
3     Random.seed!(rng, 1)
4
5     true_A = [-0.1 2.0; -2.0 -0.1]
6     true_odefunc(u, p, t) = true_A * (u .^ 3)    # nonlinear spiral (cubic term)
7
8     u0_true = Float32[2.0, 0.0]
9     tspan = (0.0f0, 1.5f0)
10    tsteps = range(tspan[1], tspan[2]; length=40)
11
12    true_prob = ODEProblem(true_odefunc, u0_true, tspan)
13    true_sol = solve(true_prob, Tsit5(); saveat=tsteps)
14    ode_data = Array(true_sol)
15 end
```

Error message

Another cell defining `tsteps` or `ode_data` contains errors.

```
1 scatter(tsteps, ode_data[1, :]; label="u1 (data)", xlabel="t", title="Observed spiral trajectory (state 1)")
```

2. Define the neural ODE ⇔

Instead of the (unknown, in a real problem) true dynamics, we parameterize f_θ with a small Lux multilayer perceptron and wrap it in an ODEProblem.

```
ComponentVector{Float32}(layer_1 = (weight = Float32[1.6523907 1.3650187; -1.1403904 -1.7166;
```

```
1 begin
2     nn_dynamics = Chain(Dense(2, 16, tanh), Dense(16, 16, tanh), Dense(16, 2))
3     ps_init, st = Lux.setup(rng, nn_dynamics)
4     ps_init = ComponentArray(ps_init)
5 end
```

```
neural_ode_func (generic function with 1 method)
```

```
1 function neural_ode_func(u, p, t)
2     û, _ = nn_dynamics(u, p, st)
3     return û
4 end
```

```
prob_neural = ODEProblem with uType Vector{Float32} and tType Float32. In-place: false
Non-trivial mass matrix: false
timespan: (0.0f0, 1.5f0)
u0: 2-element Vector{Float32}:
 2.0
 0.0
```

```
1 prob_neural = ODEProblem(neural_ode_func, u0_true, tspan)
```

3. Loss function and training loop ⇔

The loss re-solves the neural ODE with the current parameters and compares against the data. SciMLSensitivity provides the adjoint methods that make solve differentiable with respect to p, so Optimization.jl can minimize the loss with a standard gradient-based optimizer.

```
predict_neural_ode (generic function with 1 method)
```

```
1 function predict_neural_ode(p)
2     solve(prob_neural, Tsit5(); p=p, saveat=tsteps,
3         sensealg=InterpolatingAdjoint(; autojacvec=ZygoteVJP())) |> Array
4 end
```

```
loss_neural_ode (generic function with 1 method)
```

```
1 function loss_neural_ode(p, _)
2     pred = predict_neural_ode(p)
3     return sum(abs2, ode_data .- pred)
4 end
```

Error message

UndefVarError: ode_data not defined in this notebook.
Suggestion: check for spelling errors or missing imports.

Show stack trace...

```
1 begin
2     adtype = Optimization.AutoZygote()
3     optf = OptimizationFunction(loss_neural_ode, adtype)
4     optprob = OptimizationProblem(optf, ps_init)
5
6     losses = Float64[]
7     callback = function (state, loss)
8         push!(losses, loss)
9         return false
10    end
11
12    res1 = Optimization.solve(optprob, OptimizationOptimisers.Adam(0.05);
13    callback=callback, maxiters=300)
14 end
```

Error message

Another cell defining losses contains errors.

```
1 plot(losses; xlabel="iteration", ylabel="loss", yscale=:log10, label="training loss",
2 title="Neural ODE training")
```

4. Compare the learned dynamics to the ground truth ⇌

Error message

Another cell defining `res1`, `tsteps`, or `ode_data` contains errors.

```
1 begin
2     fitted = predict_neural_ode(res1.u)
3     plot(tsteps, ode_data[1, :]; label="data (u1)", lw=2, xlabel="t")
4     plot!(tsteps, fitted[1, :]; label="neural ODE (u1)", lw=2, ls=:dash)
5 end
```

Takeaways ⇌

- A neural ODE is just an `ODEProblem` whose vector field is a neural network — everything else (solvers, adaptive stepping, event handling) comes for free from the SciML solver stack.
- `SciMLSensitivity.jl` supplies the adjoint sensitivity methods that make `solve` compatible with reverse-mode autodiff (`Zygote`), which is what lets `Optimization.jl` train it like any other model.
- This idea generalizes directly to *partially* known physics — see the next notebook on Universal Differential Equations.

Next: [03_universal_differential_equations.jl](#).